Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное программное обеспечение
Дисциплина «Вычислительная математика»

Лабораторная работа №4

Вариант 17

Выполнил
Храбров Артём Алексеевич

Проверил
Наумова Надежда Александровна

Санкт-Петербург, 2026

Цель

Найти функцию, являющуюся наилучшим приближением заданной табличной функции по методу наименьших квадратов.

1. Вычислительная реализация задачи

1.1 Исходные данные

Задана функция:

$$f(x) = \frac{2x}{x^4 + 17}, \quad x \in [0, 2], \quad h = 0.2$$

Выполним табуляцию функции на интервале $[0, 2]$ с шагом $h = 0.2$.

i	x_i	y_i
1	0.0	0.00000
2	0.2	0.02353
3	0.4	0.04617
4	0.6	0.06573
5	0.8	0.07958
6	1.0	0.08333
7	1.2	0.07480
8	1.4	0.05652
9	1.6	0.03327
10	1.8	0.00761
11	2.0	-0.01961

1.2 Построение линейного приближения

Уравнение функции ищем в виде:

$$P_1(x) = ax + b$$

:

$$\begin{cases} a \sum x_i^2 + b \sum x_i = \sum x_i y_i \\ a \sum x_i + b \cdot n = \sum y_i \end{cases}$$

- $\sum x_i = 11.0$
- $\sum x_i^2 = 15.4$
- $\sum y_i = 0.46033$
- $\sum x_i y_i = 1.19762$

$$\begin{cases} 15.4a + 11.0b = 1.19762 \\ 11.0a + 11.0b = 0.46033 \end{cases}$$

$$4.4a = 0.73729 \implies a \approx 0.1676$$

$$b \approx -0.1258$$

Линейное приближение:

$$P_1(x) = 0.1676x - 0.1258$$

1.3 Построение квадратичного приближения

Уравнение функции ищем в виде:

$$P_2(x) = ax^2 + bx + c$$

$$\begin{cases} a\sum x_i^4 + b\sum x_i^3 + c\sum x_i^2 = \sum x_i^2 y_i \\ a\sum x_i^3 + b\sum x_i^2 + c\sum x_i = \sum x_i y_i \\ a\sum x_i^2 + b\sum x_i + c \cdot n = \sum y_i \end{cases}$$

- $\sum x_i^3 = 39.6$
- $\sum x_i^4 = 110.0$
- $\sum x_i^2 y_i = 0.94919$

$$\begin{cases} 110.0a + 39.6b + 15.4c = 0.94919 \\ 39.6a + 15.4b + 11.0c = 1.19762 \\ 15.4a + 11.0b + 11.0c = 0.46033 \end{cases}$$

$$a \approx -0.0983, \quad b \approx 0.3392, \quad c \approx -0.0136$$

Квадратичное приближение:

$$P_2(x) = -0.0983x^2 + 0.3392x - 0.0136$$

1.4 Нахождение среднеквадратических отклонений

СКО рассчитывается по формуле:

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - P(x_i))^2}$$

Для линейной аппроксимации:

Вычислив сумму квадратов разностей $\sum(y_i - P_1(x_i))^2$, получаем $S_1 = 0.00835$:

$$\sigma_1 = \sqrt{\frac{0.00835}{11}} = \sqrt{0.000759} \approx 0.0276$$

Для квадратичной аппроксимации:

Сумма квадратов разностей $\sum(y_i - P_2(x_i))^2$ составляет $S_2 = 0.00173$:

$$\sigma_2 = \sqrt{\frac{0.00173}{11}} = \sqrt{0.000157} \approx 0.0125$$

1.5 Сравнение результатов

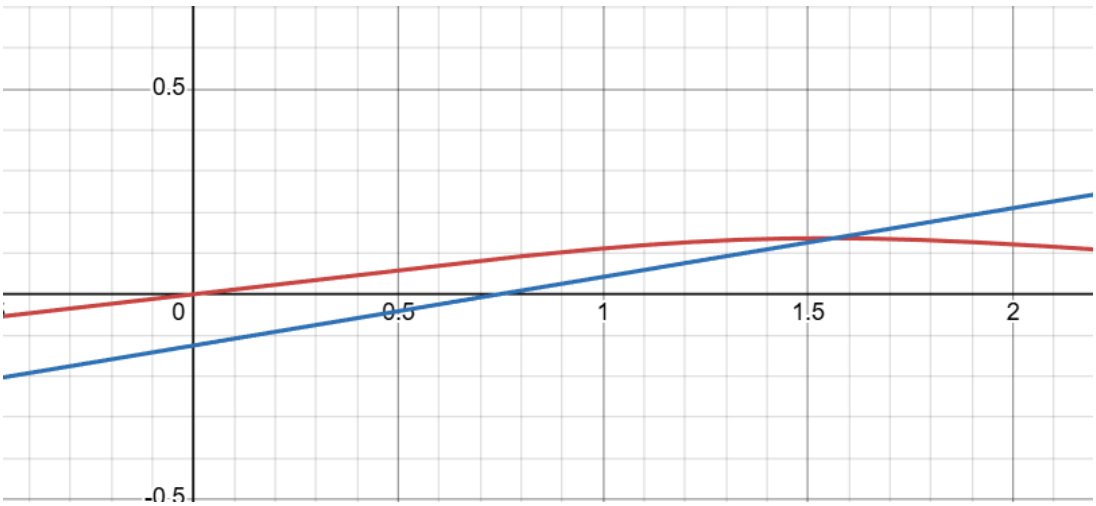
$$\sigma_2(0.0125) < \sigma_1(0.0276)$$

Вывод: Квадратичное приближение $P_2(x) = -0.0983x^2 + 0.3392x - 0.0136$ описывает заданный набор точек точнее, поэтому оно является наилучшим.

2. Графики

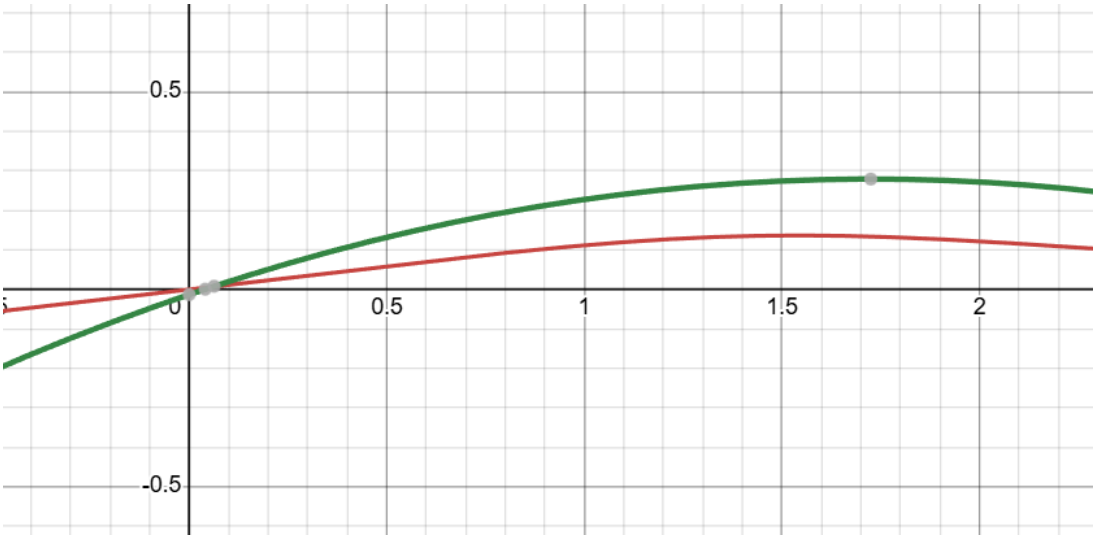
2.1 Линейное приближение

$$f(x) = \frac{2x}{x^4 + 17}, \quad g(x) = 0.1676x - 0.1258$$

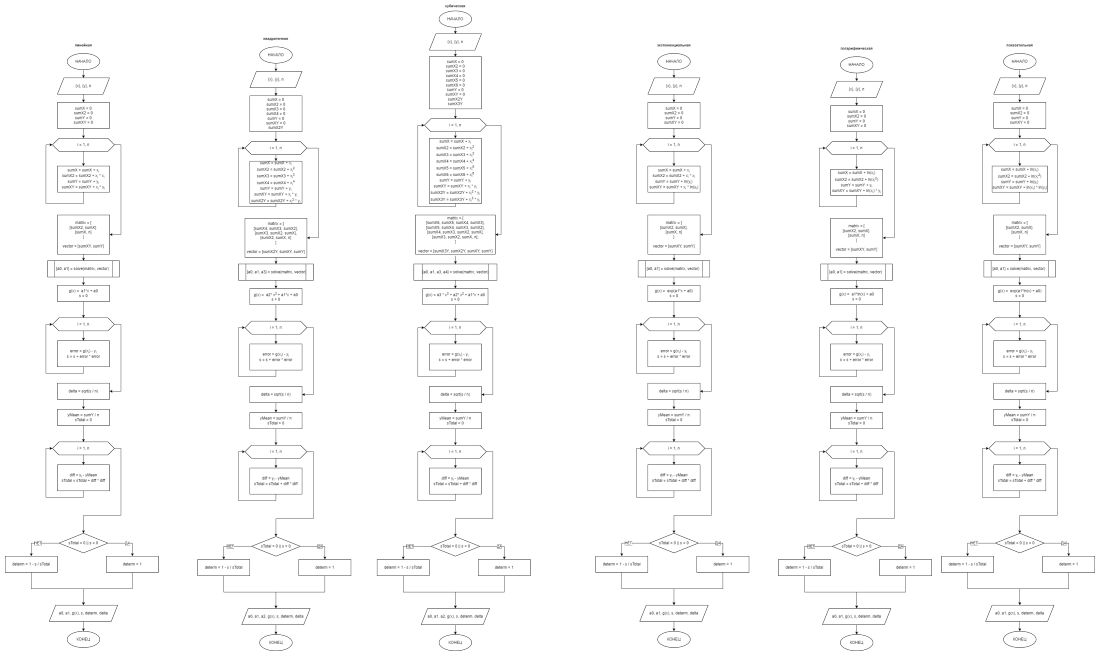


2.2 Квадратичное приближение

$$f(x) = \frac{2x}{x^4 + 17}, \quad g(x) = -0.0983x^2 + 0.3392x - 0.0136$$



3. Блок-схема



4. Листинг программы

```
package arhr.tech.comp_math_lab2.services.solvers

import arhr.tech.comp_math_lab2.dto.ApproximationRequestDto
import arhr.tech.comp_math_lab2.dto.ApproximationResult
import arhr.tech.comp_math_lab2.dto.PointDto
import arhr.tech.comp_math_lab2.utils.SystemSolver
import java.math.BigDecimal
import java.math.MathContext
import java.math.RoundingMode
import kotlin.math.sqrt


abstract class AbstractApproximationSolver(
    protected val type: ApproximationType
) : ApproximationSolver {

    protected val mc = MathContext.DECIMAL64
    protected val systemSolver = SystemSolver()

    protected abstract fun buildMatrix(points: List<PointDto>): Array<
   Array<BigDecimal>>

    protected abstract fun buildVector(points: List<PointDto>): Array<
   BigDecimal>

```

```
    protected abstract fun evaluate(coefficients: List<BigDecimal>, x: BigDecimal): BigDecimal

    protected abstract fun buildFormula(coefficients: List<BigDecimal>): String

    override fun solve(request: ApproximationRequestDto): ApproximationResult {
        val points = request.points
        val n = points.size.toBigDecimal()

        //
        val matrix = buildMatrix(points)
        val vector = buildVector(points)
        val solution = systemSolver.solve(matrix, vector)
        val coefficients = solution.toList()

        //
        val func = buildFormula(coefficients)

        // S
        var s = BigDecimal.ZERO
        for (point in points) {
            val yPred = evaluate(coefficients, point.x)
            val error = yPred - point.y
            s += error * error
        }

        //
        val delta = if (s != BigDecimal.ZERO) {
            sqrt(s.divide(n, mc).toDouble()).toBigDecimal()
        } else {
            BigDecimal.ZERO
        }

        // determ
        val sumY = points.sumOf { it.y }
        val yMean = sumY.divide(n, mc)

        var sTotal = BigDecimal.ZERO
        for (point in points) {
            val diff = point.y - yMean
            sTotal += diff * diff
        }

        val determ = when {
            sTotal == BigDecimal.ZERO -> BigDecimal.ONE
            s == BigDecimal.ZERO -> BigDecimal.ONE
```

```
            else -> BigDecimal.ONE - s.divide(sTotal, mc)
        }

        // pearson
        val pearson = if (type == ApproximationType.LINEAR) {
            val sumX = points.sumOf { it.x }
            val xMean = sumX.divide(n, mc)

            var sumXdevYdev = BigDecimal.ZERO
            var sumXdev2 = BigDecimal.ZERO
            var sumYdev2 = BigDecimal.ZERO

            for (point in points) {
                val xDev = point.x - xMean
                val yDev = point.y - yMean
                sumXdevYdev += xDev * yDev
                sumXdev2 += xDev * xDev
                sumYdev2 += yDev * yDev
            }

            if (sumXdev2 != BigDecimal.ZERO && sumYdev2 != BigDecimal.ZERO) {
                sumXdevYdev.divide(
                    sqrt((sumXdev2 * sumYdev2).toDouble()).toBigDecimal(),
                    mc
                )
            } else {
                BigDecimal.ZERO
            }
        } else {
            null
        }

        return ApproximationResult(
            type = type,
            func = func,
            s = s.round(mc),
            determ = determ.round(mc),
            delta = delta.round(mc),
            pearson = pearson?.round(mc),
            coffs = coefficients.map { it.round(mc) }
        )
    }

    override fun supports(type: ApproximationType): Boolean = this.type == type

```

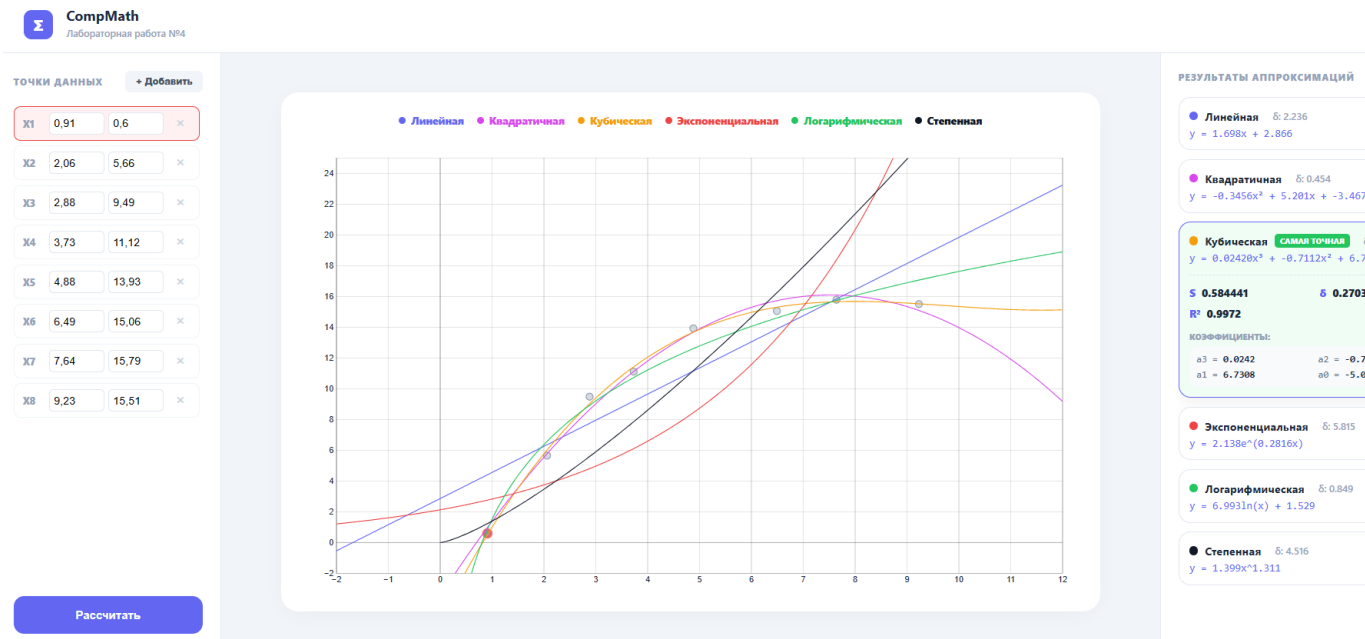
```

    protected fun BigDecimal.round(): BigDecimal {
        if (this.abs() <= BigDecimal("0.0000001")) return BigDecimal.ZERO
        return this.round(MathContext(4, RoundingMode.HALF_UP))
    }

    protected fun List<BigDecimal>.sumOf(selector: (BigDecimal) -> BigDecimal): BigDecimal {
        var result = BigDecimal.ZERO
        for (element in this) {
            result += selector(element)
        }
        return result
    }

    protected fun Int.pow(exp: Int): Int {
        var result = 1
        repeat(exp) { result *= this }
        return result
    }
}
```

5. Пример работы программы



Вывод

В ходе выполнения данной работы я изучил метод наименьших квадратов и реализовал его на языке Kotlin.Нашёл аппроксимации функции $f(x) = \frac{2x}{x^4+17}$. Квадратичное приближение оказалось точнее линейного.